Zipf-biased Sampling Utilities

Explaining _zipf_pmf and _zipf_sample with math, examples, and MATLAB equivalents.

Code in scope

The following functions build a Zipf probability mass function (PMF) over ranks and then sample one element from a ranked list. The seeding helper is intentionally excluded here.

```
def _zipf_pmf(n: int, s: float) -> np.ndarray:
    """Return normalized Zipf PMF over ranks 1..n for exponent s."""
    ranks = np.arange(1, n + 1, dtype=float)
    pmf = 1.0 / ranks**s
    pmf /= pmf.sum()
    return pmf

def _zipf_sample(pool: List, s: float):
    """Sample one element from a ranked pool according to Zipf(s)."""
    pmf = _zipf_pmf(len(pool), s)
    idx = np.random.choice(len(pool), p=pmf)
    return pool[idx]
```

1) What problem this solves

A ranked pool is common in security and usability studies: some choices are more popular than others. Instead of sampling uniformly, Zipf sampling assigns higher probability to top ranks (earlier list positions).

```
Rank 1 (pool[0])  -> most likely
Rank 2 (pool[1])  -> next most likely
...
Rank n (pool[n-1]) -> least likely
```

2) The Zipf PMF in mathematics

Let ranks r be 1..n and let s be the Zipf exponent.

```
Weights:       w(r) = 1 / r^s
Normalizer:     H(n,s) = sum_{k=1..n} 1 / k^s   (generalized harmonic number)
PMF:            p(r) = w(r) / H(n,s) = (1 / r^s) / H(n,s)
```

Interpretation of s: s=0 produces a uniform distribution. Larger s concentrates probability on small ranks (hotspots).

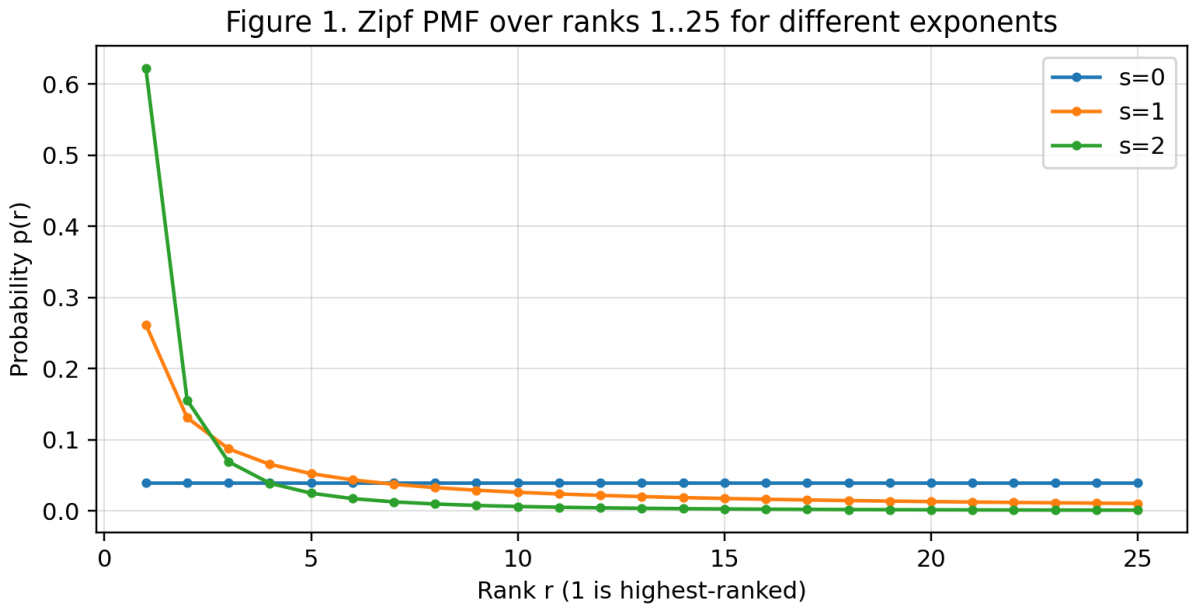

Figure 1 shows how probability mass shifts toward rank 1 as s increases.

3) Mapping ranks to Python indices

Python lists are 0-indexed, but Zipf ranks are typically written 1..n.

```
Rank:     1      2      3      ...     n
Index:     0      1      2      ...     n-1
Pool:   pool[0] pool[1] pool[2] ... pool[n-1]
```

_zipf_pmf returns an array pmf of length n where pmf[i] corresponds to rank r=i+1. _zipf_sample draws an index i using pmf, then returns pool[i].

4) Worked numeric example

Example pool: [A, B, C, D, E] is treated as ranks 1..5.

Rank r	Pool element	p(r) (s=0)	p(r) (s=1)	p(r) (s=2)
1	A	0.2000	0.4380	0.6832
2	B	0.2000	0.2190	0.1708
3	C	0.2000	0.1460	0.0759
4	D	0.2000	0.1095	0.0427
5	E	0.2000	0.0876	0.0273

In this example, s=2 heavily favors A (rank 1). If the pool represents password choices ranked by popularity, Zipf sampling simulates a strong hotspot on the most popular choices.

5) Sampling behavior: expected vs observed

Sampling repeatedly from the PMF produces empirical frequencies close to the theoretical probabilities. Figure 2 compares the PMF to frequencies from 20,000 draws (n=12, s=2).

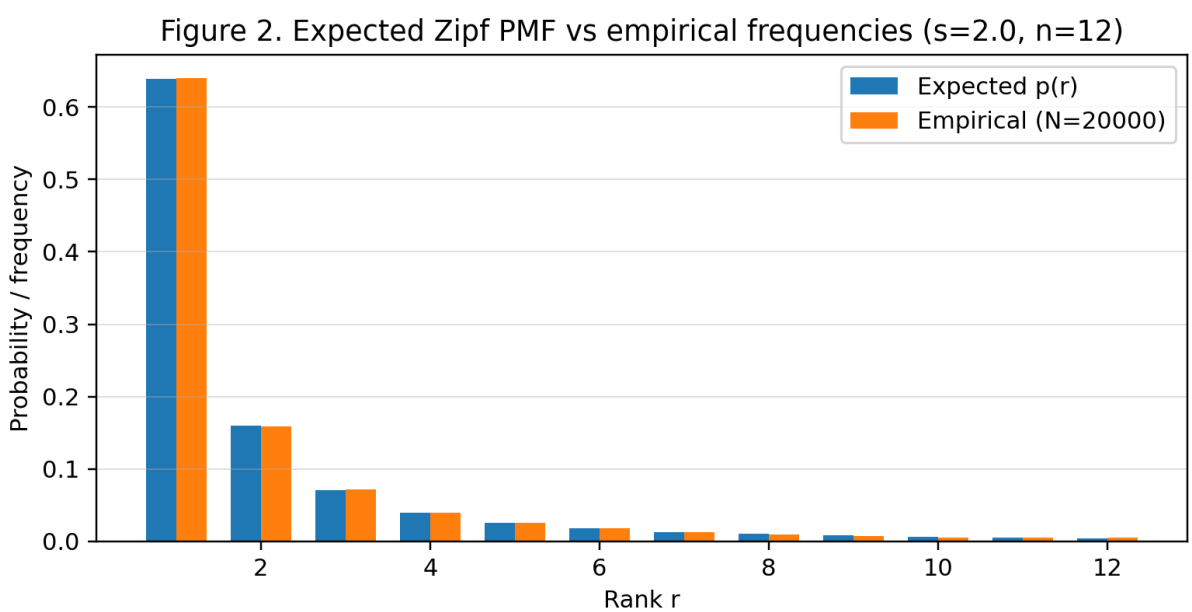

Figure 2. Bars should match closely as the number of draws increases.

6) How sampling works internally (CDF method)

np.random.choice uses the provided probabilities to select an index. Conceptually, one common implementation is the CDF method: draw u uniformly in [0,1) and choose the smallest rank r whose cumulative probability is at least u.

```
CDF method (conceptual):
1) Compute cumulative sums: cdf[r] = sum_{k=1..r} p(k)
2) Draw u ~ Uniform(0,1)
3) Return smallest r with cdf[r] >= u
```

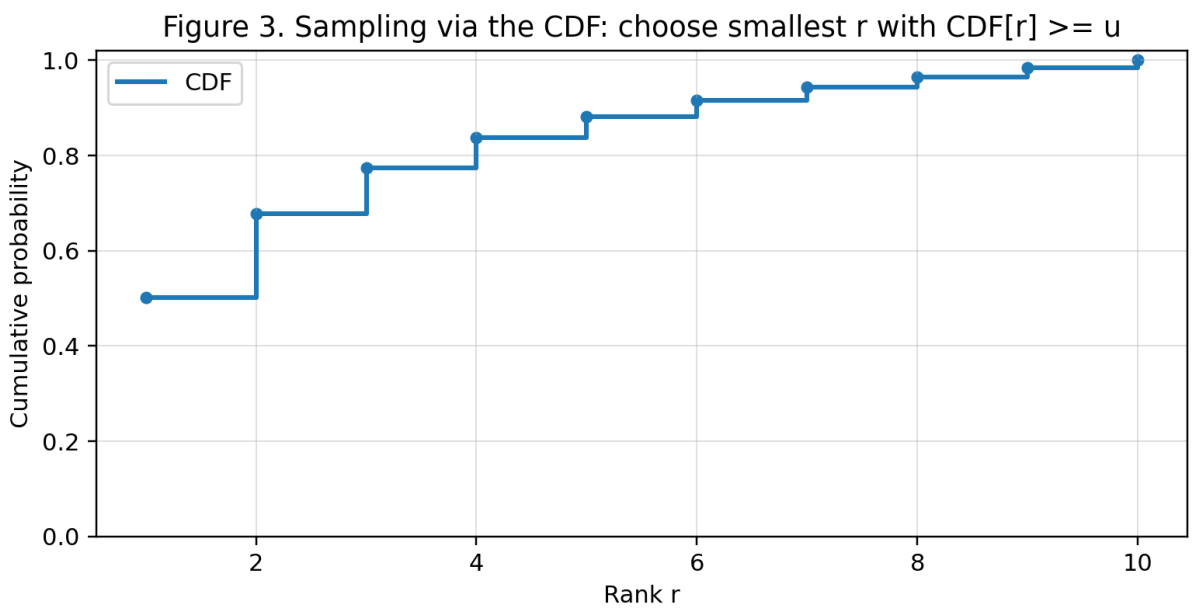

Figure 3. A uniform draw u selects an interval of the CDF, which maps to a rank.

7) Example end-to-end flow (pseudocode)

```
INPUT: ranked pool, exponent s

pmf = zipf_pmf(n=len(pool), s)

# sample one element
idx = random_choice_index(0..n-1, probabilities=pmf)
chosen = pool[idx]

RETURN chosen
```

8) MATLAB equivalent (for MATLAB-first readers)

The following MATLAB code mirrors the Python logic. It includes two sampling options: (A) using randsample with weights (Statistics and Machine Learning Toolbox) and (B) a toolbox-free CDF method.

```
% ---------- zipf_pmf.m ----------
function pmf = zipf_pmf(n, s)
    r = (1:n)';                 % ranks 1..n as a column vector
    w = 1 ./ (r .^ s);          % weights: 1/r^s
    pmf = w ./ sum(w);          % normalize to sum to 1
end

% ---------- zipf_sample.m ----------
% pool can be a numeric vector or a cell array of strings/objects
function item = zipf_sample(pool, s)
    n = numel(pool);
    pmf = zipf_pmf(n, s);

    % Option A (requires randsample):
    % idx = randsample(n, 1, true, pmf);

    % Option B (toolbox-free): CDF method
    cdf = cumsum(pmf);
    u = rand();
    idx = find(cdf >= u, 1, 'first');

    item = pool(idx);
end
```

MATLAB usage example

```
pool = {'A','B','C','D','E'};
s = 2.0;

% Draw 10 samples
for t = 1:10
    x = zipf_sample(pool, s);
    disp(x{1});
end
```

Key mapping note

MATLAB indices are 1-based, so rank r corresponds directly to index r. In Python, rank r corresponds to index r-1.